

IF2224 TEORI BAHASA FORMAL DAN OTOMATA

TUGAS BESAR

Milestone 3

(Kelompok TBFO)



Dipersiapkan oleh:

13524044	Narendra Dharma Wistara M
13524060	Steven Tan
13524090	Nashiruddin Akram
13524100	Arghawisesa Dwinanda Arham

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026**

Daftar Isi

Daftar Isi.....	1
Teori Singkat.....	1
A. Semantic Analysis.....	2
B. Abstract Syntax Tree.....	2
C. Decorated Abstract Syntax Tree.....	3
D. Attributed Grammar dan Syntax-Directed Translation.....	3
E. Symbol Table.....	4
F. Scope dan Lexical Level.....	4
G. Type Checking dan Type Compatibility.....	5
Perancangan & Implementasi.....	6
A. Teknologi yang Digunakan.....	6
B. Gambaran Umum Arsitektur.....	6
C. Struktur Program.....	6
D. Fungsi/Kelas Utama.....	8
E. Alur Kerja Program.....	11
F. Implementasi AST.....	12
G. Implementasi Symbol Table.....	14
H. Implementasi Semantic Analyzer.....	16
I. Implementasi Error Handling & Output.....	21
Pengujian.....	24
Pengujian 1.....	24
Pengujian 2.....	27
Pengujian 3.....	30
Pengujian 4.....	32
Pengujian 5.....	35
Kesimpulan dan Saran.....	39
Kesimpulan.....	39
Saran.....	40
Lampiran.....	41
Referensi.....	42

Teori Singkat

Pada proses kompilasi, sebuah program sumber tidak cukup hanya diperiksa berdasarkan sintaksisnya saja. Setelah tahap *lexical analysis* menghasilkan token dan tahap *syntax analysis* menghasilkan *parse tree*, *compiler* masih perlu memastikan bahwa program tersebut memiliki makna semantik yang valid. Pada Milestone 3 Arion Compiler, dibangun *semantic analysis* yang bertugas menelusuri hasil *parse tree* dari milestone sebelumnya, membangun struktur AST, melakukan pengecekan tipe dan lingkup, serta menghasilkan *decorated AST* dan *symbol table*.

A. Semantic Analysis

Semantic analysis adalah tahap *compiler* yang memeriksa kebenaran makna suatu program. Berbeda dengan *syntax analysis* yang hanya memastikan susunan token mengikuti grammar, *semantic analysis* memastikan bahwa setiap operasi, deklarasi, pemanggilan fungsi/prosedur, dan penggunaan identifier sesuai dengan aturan semantik bahasa.

Beberapa pengecekan utama dalam semantic analysis adalah sebagai berikut.

1. *type checking*, yaitu proses memastikan bahwa operator dan operand memiliki tipe data yang kompatibel. Sebagai contoh, operasi aritmatika seperti penjumlahan hanya valid untuk tipe numerik tertentu, sedangkan ekspresi pada kondisi *if* atau *while* harus menghasilkan nilai bertipe Boolean.
2. *symbol table management*, yaitu pengelolaan informasi identifier yang muncul di dalam program. Identifier seperti variabel, konstanta, tipe, fungsi, dan prosedur harus dicatat dalam *symbol table* agar compiler dapat mengetahui apakah sebuah identifier sudah dideklarasikan sebelum digunakan.
3. *scope resolution*, yaitu proses menentukan apakah suatu identifier dapat diakses dari posisi tertentu dalam program. Hal ini penting karena identifier yang dideklarasikan pada scope lokal tidak selalu dapat diakses dari scope lain.
4. *control-flow validation*, yaitu pengecekan semantik terhadap struktur alur kontrol. Contohnya adalah memastikan bahwa instruksi tertentu digunakan pada konteks yang benar dan bahwa struktur program tidak menghasilkan alur eksekusi yang tidak valid.

B. Abstract Syntax Tree

Abstract Syntax Tree atau AST adalah representasi pohon dari program yang lebih ringkas dibandingkan *parse tree*. *Parse tree* masih memuat detail grammar secara lengkap, termasuk

simbol terminal, nonterminal, tanda baca, dan keyword yang diperlukan untuk membuktikan bahwa program valid secara sintaksis. Namun, tidak semua informasi tersebut diperlukan pada tahap *semantic analysis*.

AST hanya menyimpan informasi yang penting untuk analisis makna program. Misalnya, untuk ekspresi $a + 10$, AST cukup merepresentasikan operasi $+$ dengan dua operand, yaitu variabel a dan konstanta 10 . Detail sintaksis seperti aturan turunan grammar, tanda kurung yang tidak diperlukan, atau simbol pemisah dapat dihilangkan. Dengan demikian, AST memudahkan compiler untuk memahami hubungan semantik antarbagian program, seperti hubungan antara operator dan operand, target dan nilai pada assignment, serta nama prosedur dan argumennya.

C. Decorated Abstract Syntax Tree

Decorated AST adalah AST yang telah diberi anotasi tambahan hasil dari proses *semantic analysis*. Anotasi tersebut dapat berupa tipe data, indeks atau referensi ke symbol table, lexical level, informasi scope, serta status validitas semantik suatu node. Sebagai contoh, sebuah node `VarNode("a")` pada AST awal hanya menyimpan nama variabel a . Setelah proses dekorasi, node tersebut dapat memiliki informasi tambahan seperti tipe `integer`, indeks identifier pada `tab`, dan level scope tempat variabel tersebut dideklarasikan. Dengan adanya informasi ini, AST tidak hanya menggambarkan struktur program, tetapi juga membawa informasi semantik yang diperlukan untuk tahap compiler berikutnya.

D. Attributed Grammar dan Syntax-Directed Translation

Attributed Grammar adalah *grammar* yang dilengkapi dengan atribut dan aturan semantik. Atribut digunakan untuk menyimpan informasi tambahan pada *node grammar*, seperti tipe data, nilai konstanta, hasil ekspresi, atau referensi ke *symbol table*. Aturan semantik menentukan bagaimana atribut tersebut dihitung berdasarkan atribut dari *node* lain.

Pada implementasi *semantic analysis*, pendekatan ini digunakan melalui *Syntax-Directed Translation*, yaitu proses menerjemahkan struktur sintaksis menjadi struktur semantik dengan menyisipkan aksi semantik pada aturan produksi grammar. Aksi semantik tersebut dapat digunakan untuk membentuk node AST, menghitung tipe ekspresi, atau melakukan validasi terhadap aturan bahasa.

E. Symbol Table

Symbol table adalah struktur data yang digunakan compiler untuk menyimpan informasi mengenai identifier dalam program. Identifier dapat berupa nama program, variabel, konstanta, tipe data, prosedur, fungsi, parameter, maupun field record. Informasi yang disimpan biasanya meliputi nama identifier, jenis objek, tipe data, scope, alamat relatif, serta referensi ke struktur lain jika identifier memiliki tipe kompleks.

Dalam program ini diimplementasikan tiga *symbol table*, yaitu:

1. *tab* digunakan untuk menyimpan informasi utama identifier, seperti nama identifier, jenis objek, tipe data, lexical level, alamat, dan link ke identifier lain dalam scope yang sama.
2. *btabs* digunakan untuk menyimpan informasi block, seperti block untuk program, prosedur, fungsi, atau record. Tabel ini menyimpan informasi seperti identifier terakhir dalam block, parameter terakhir, ukuran parameter, dan ukuran variabel lokal.
3. *atabs* digunakan untuk menyimpan informasi tipe array, seperti tipe indeks, batas bawah dan atas indeks, tipe elemen, ukuran elemen, serta ukuran total array.

Dengan adanya symbol table, semantic analyzer dapat memastikan bahwa identifier sudah dideklarasikan sebelum digunakan, tidak terjadi deklarasi ulang pada scope yang sama, dan setiap ekspresi menggunakan tipe data yang sesuai.

F. Scope dan Lexical Level

Scope adalah wilayah dalam program tempat suatu identifier dapat dikenali dan digunakan. Identifier yang dideklarasikan pada scope global dapat diakses dari banyak bagian program, sedangkan identifier lokal hanya berlaku di dalam block tertentu. Untuk mengelola hal ini, compiler menggunakan konsep *lexical level*, yaitu tingkat kedalaman suatu block dalam struktur program.

Pada saat semantic analyzer memasuki block baru, misalnya prosedur, fungsi, atau compound statement tertentu, analyzer dapat melakukan *push* terhadap informasi scope baru. Ketika keluar dari block tersebut, analyzer melakukan *pop* agar identifier lokal tidak lagi dianggap aktif. Mekanisme ini memungkinkan lookup identifier dilakukan dari scope terdalam menuju scope terluar.

Sebagai contoh, jika sebuah variabel digunakan dalam suatu *block*, *semantic analyzer* akan mencari deklarasi variabel tersebut mulai dari *scope* lokal. Jika tidak ditemukan, pencarian

dilanjutkan ke *scope* di luarnya hingga mencapai *scope global*. Jika identifier tetap tidak ditemukan, maka terjadi semantic error karena identifier digunakan tanpa deklarasi.

G. Type Checking dan Type Compatibility

Type checking adalah proses memeriksa apakah tipe data yang digunakan dalam suatu operasi sudah sesuai dengan aturan bahasa. Setiap ekspresi akan memiliki tipe hasil tertentu. Misalnya, ekspresi aritmetika antara dua integer menghasilkan integer, ekspresi relasional menghasilkan Boolean, dan literal string memiliki tipe String. Sedangkan, *type compatibility* menentukan apakah dua tipe data boleh digunakan bersama dalam suatu konteks. Pada assignment statement, misalnya, tipe ekspresi di sebelah kanan harus kompatibel dengan tipe variabel di sebelah kiri. Jika variabel bertipe Integer diberi nilai Boolean, maka assignment tersebut tidak valid secara semantik.

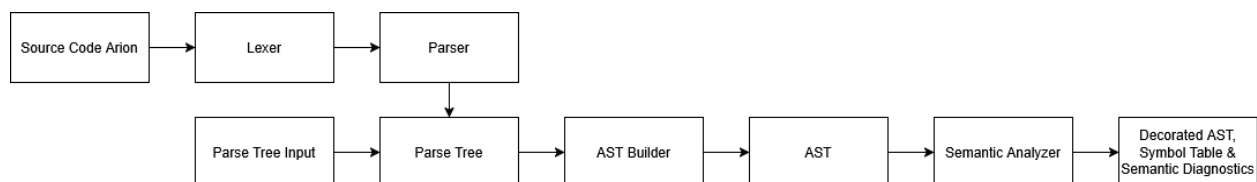
Perancangan & Implementasi

A. Teknologi yang Digunakan

Program diimplementasikan menggunakan bahasa C++17 dengan bantuan Standard Library seperti `std::vector`, `std::string`, `std::ostringstream`, dan exception handling. Makefile digunakan untuk mendukung proses kompilasi agar seluruh file sumber dapat dibangun secara konsisten. Program dijalankan melalui terminal dengan input berupa source code Arion atau parse tree, dan output dapat ditampilkan ke terminal atau ditulis ke file.

B. Gambaran Umum Arsitektur

Program dirancang sebagai pipeline compiler yang modular. Secara umum, input berupa source code Arion diproses melalui lexer dan parser untuk menghasilkan parse tree. Selain itu, program juga dapat menerima input berupa parse tree secara langsung. Setelah parse tree-nya telah dibuat, tahap Milestone 3 dimulai dengan konversi parse tree menjadi AST, kemudian AST dianalisis oleh semantic analyzer dengan bantuan Symbol Table.



Sesuai diagram di atas, lexer dan parser berperan sebagai tahap pendukung dari milestone sebelumnya untuk menghasilkan parse tree dari source code. Jika input sudah berupa parse tree, program dapat langsung melanjutkan ke tahap AST builder. Setelah itu, AST yang terbentuk dianalisis oleh semantic analyzer untuk menghasilkan Decorated AST, Symbol Table, dan Semantic Diagnostics.

C. Struktur Program

Program disusun secara modular dengan memisahkan setiap tahap pemrosesan ke dalam file yang berbeda. Struktur utama program sebagai berikut:

```
ISL-Tubes-IF2224-2026/
├── src/
│   ├── main.cpp
│   ├── token.h
│   ├── lexer.h / lexer.cpp
│   ├── parser.h / parser.cpp
│   └── parse_tree.h / parse_tree.cpp
```

```

| └─ ast.h / ast.cpp
| └─ ast_builder.h / ast_builder.cpp
| └─ symbol_table.h / symbol_table.cpp
| └─ semantic_analyzer.h / semantic_analyzer.cpp
└─ Makefile

```

Tabel Modul-Modul Program

Modul	Tanggung Jawab
main.cpp	Mengatur alur eksekusi program, mulai dari membaca input, menjalankan lexer/parser jika diperlukan, membangun AST, menjalankan semantic analyzer, sampai menghasilkan output.
token.h	Mendefinisikan jenis token dan struct token yang digunakan oleh lexer dan parser.
lexer.h / lexer.cpp	Melakukan lexical analysis untuk mengubah source code Arion menjadi daftar token.
parser.h / parser.cpp	Melakukan syntax analysis dengan recursive descent dan menghasilkan parse tree.
parse_tree.h / parse_tree.cpp	Menyimpan struktur parse tree serta fungsi untuk membaca atau menampilkan parse tree.
ast.h / ast.cpp	Mendefinisikan struct AST dan fungsi untuk menampilkan AST, termasuk AST yang sudah diberi anotasi semantic.
ast_builder.h / ast_builder.cpp	Mengubah parse tree menjadi AST yang lebih ringkas untuk semantic analysis.
symbol_table.h / symbol_table.cpp	Mengelola Symbol Table, termasuk tab, btab, atab, scope, predefined identifier, insert, lookup, push, dan pop.
semantic_analyzer.h / semantic_analyzer.cpp	Melakukan semantic analysis dengan menelusuri AST, mengecek aturan semantic, memperbarui Symbol Table, dan memberi anotasi pada AST.
Makefile	Mengatur proses kompilasi seluruh file sumber agar program dapat di- <i>build</i> secara konsisten dan praktis.

D. Fungsi/Kelas Utama

Pada bagian ini, komponen utama program dibagi menjadi dua kelompok, yaitu komponen data dan komponen proses. Komponen data berupa class, struct, dan enum digunakan untuk menyimpan representasi program, seperti parse tree, AST, Symbol Table, dan hasil semantic analysis. Komponen proses berupa fungsi dan method digunakan untuk menjalankan konversi parse tree menjadi AST, semantic analysis, pengelolaan Symbol Table, dan pembentukan output.

Class, struct, dan enum pada program digunakan sebagai representasi data utama selama proses semantic analysis berjalan.

Tabel Kelas-Kelas Utama pada Tahap Semantic Analyzer

Komponen	Tipe	Peran Singkat
ParseNode	Struct	Merepresentasikan node pada parse tree.
AstKind	Enum class	Membedakan jenis node pada AST.
AstNode	Struct	Merepresentasikan node AST dan menyimpan anotasi semantic.
ObjectKind	Enum class	Membedakan jenis objek pada Symbol Table.
TypeKind	Enum class	Membedakan jenis tipe data yang dikenali semantic analyzer.
BlockKind	Enum class	Membedakan jenis block atau scope.
TabEntry	Struct	Menyimpan satu entri identifier pada tab.
BTabEntry	Struct	Menyimpan informasi block atau scope pada btab.

ATabEntry	Struct	Menyimpan informasi array pada atab.
SymbolTable	Class	Mengelola Symbol Table, scope, predefined identifier, dan output tabel.
TypeInfo	Helper Struct	Menyimpan informasi tipe sementara selama semantic analysis.
ExprInfo	Helper Struct	Menyimpan hasil evaluasi semantic dari expression.
SemanticAnalyzer	Helper Class	Menjalankan traversal AST, pengecekan semantic, anotasi AST, dan pengumpulan diagnostics.
SemanticDiagnostic	Struct	Menyimpan satu pesan error atau warning semantic.
SemanticResult	Struct	Menyimpan Decorated AST, Symbol Table, dan diagnostics.

Fungsi dan method di bawah ini dipilih karena bertindak sebagai pelaksana pemrosesan yang diperlukan untuk tahap semantic analysis.

Tabel Fungsi-Fungsi & Method-Method Utama pada Tahap Semantic Analyzer

Fungsi / Method	Peran Singkat
main	Mengatur pipeline utama dari input sampai output.
looksLikeParseTreeInput	Mengecek apakah input terlihat seperti parse tree.
parseRenderedParseTree	Mengubah teks parse tree menjadi ParseNode.
buildAstFromParseTree	Mengubah parse tree menjadi AST.
analyzeSemantics	Menjalankan semantic analysis dari luar SemanticAnalyzer.
SemanticAnalyzer::analyze	Memulai traversal semantic analyzer dan menghasilkan SemanticResult.

annotate	Memberi anotasi semantic pada node AST.
insertSymbol	Menambahkan simbol ke Symbol Table dan menangani error insert.
visitProgram	Memulai traversal dari root program.
visitDeclarations	Memproses deklarasi const, type, var, procedure, dan function.
visitParameters	Memproses parameter subprogram dan memasukkannya ke Symbol Table.
resolveType	Menentukan tipe dari node tipe.
resolveArrayType	Membentuk informasi tipe array dan menyimpannya ke atab.
resolveRecordType	Membentuk scope record dan field record.
visitStatement	Mengarahkan pengecekan statement berdasarkan jenisnya.
visitAssignment	Mengecek target, nilai, dan kompatibilitas assignment.
evalExpression	Mengevaluasi expression dan menghasilkan informasi semantic.
evalIdentifier	Mengecek penggunaan identifier dan informasi semantic terkait.
evalVariable	Mengevaluasi variable beserta component variable.
evalIndexAccess	Mengecek akses array.
evalFieldAccess	Mengecek akses field record.
evalCall	Mengecek pemanggilan procedure atau function.
checkCallArguments	Mengecek jumlah dan tipe argumen call.
typeCompatible	Mengecek kompatibilitas tipe secara umum.
assignmentCompatible	Mengecek kompatibilitas nilai dengan target assignment atau parameter.
requireBoolean	Memastikan expression kondisi bertipe boolean.

<code>SemanticResult::ok</code>	Menentukan apakah hasil semantic analysis bebas dari error.
<code>SymbolTable::initialize</code>	Mengisi Symbol Table awal.
<code>SymbolTable::insert</code>	Menambahkan identifier ke tab.
<code>SymbolTable::lookup</code>	Mencari identifier dari scope terdalam ke luar.
<code>SymbolTable::pushScope / popScope</code>	Mengelola masuk dan keluar scope.
<code>SymbolTable::insertArray</code>	Menambahkan informasi array ke atab.
<code>SymbolTable::renderAll</code>	Menghasilkan output tab, btab, dan atab.
<code>renderAst</code>	Menampilkan AST beserta anotasi semantic.

E. Alur Kerja Program

Program dijalankan melalui terminal dengan satu file input dan satu nama file output yang bersifat opsional. File input dapat berupa source code Arion atau parse tree hasil parser. Jika file output tidak diberikan, hasil analisis ditampilkan hanya ke terminal. Jika file output diberikan, hasil analisis akan ditulis juga ke file tersebut.

Alur kerja program adalah sebagai berikut.

1. Validasi argumen untuk main

Program memeriksa jumlah argumen yang diberikan. Argumen wajib berisi path file input, sedangkan path file output bersifat opsional.

2. Pembacaan file input

Program membaca seluruh isi file input terlebih dahulu sebelum menentukan tahap pemrosesan berikutnya.

3. Deteksi jenis input

Jika isi file diawali dengan node <program>, input dianggap sebagai parse tree. Jika tidak, input dianggap sebagai source code Arion.

4. Pemrosesan source code atau parse tree

Jika input berupa source code, program menjalankan lexer untuk menghasilkan token, lalu parser untuk menghasilkan parse tree. Jika input sudah berupa parse tree, program langsung membentuk struct ParseNode dari input tersebut.

5. **Konversi parse tree menjadi AST**

Setelah parse tree tersedia, program menjalankan AST builder untuk membentuk AST. Tahap ini diperlukan karena AST lebih ringkas dan lebih sesuai untuk proses semantic analysis dibandingkan parse tree.

6. **Semantic analysis**

AST diberikan ke fungsi `analyzeSemantics`. Pada tahap ini, semantic analyzer menelusuri AST, menggunakan dan memperbarui Symbol Table, melakukan pengecekan semantic, serta memberi anotasi pada node AST. Hasil tahap ini disimpan dalam `SemanticResult`.

7. **Pembentukan hasil analisis**

`SemanticResult` berisi Decorated AST, Symbol Table, dan Semantic Diagnostics. Decorated AST menunjukkan AST yang sudah diberi anotasi semantic, Symbol Table berisi informasi identifier dan scope, sedangkan Semantic Diagnostics berisi error atau warning yang ditemukan.

8. **Output dan status program**

Hasil analisis ditampilkan ke terminal atau ditulis ke file output. Jika terdapat semantic error, program tetap menampilkan hasil analisis, tetapi mengembalikan status gagal.

Alur ini dipilih agar program dapat digunakan dalam dua cara. Pertama, program dapat dijalankan dengan input source code Arion. Kedua, program tetap dapat menerima parse tree secara langsung. Dengan begitu, pengujian dapat dilakukan baik dari source code maupun dari parse tree yang sudah disiapkan.

F. Implementasi AST

Pada Milestone 3, parse tree dari tahap parser tidak langsung digunakan untuk semantic analysis. Parse tree masih menyimpan banyak detail sintaks seperti keyword, tanda baca, dan bentuk grammar yang lengkap. Oleh karena itu, parse tree dikonversi terlebih dahulu menjadi **Abstract Syntax Tree (AST)** yang lebih ringkas.

Konversi parse tree menjadi AST pada program ini menggunakan pendekatan **Syntax-Directed Translation**. Dalam implementasi, semantic action tidak ditulis langsung di dalam aturan grammar, tetapi diwujudkan melalui fungsi-fungsi pada `ast_builder.cpp`. Fungsi AST builder membaca node parse tree, memilih informasi yang masih diperlukan, lalu membentuk node AST yang lebih sesuai untuk semantic analysis.

AST dirancang menggunakan `AstNode` dan `AstKind`. `AstKind` digunakan untuk membedakan jenis node, seperti `Program`, `VarDecl`, `AssignStmt`, `IfStmt`, `Call`,

Variable, BinaryExpr, dan Literal. Sementara itu, AstNode menyimpan jenis node, teks node, dan daftar child node. Dengan bentuk ini, semantic analyzer dapat memproses AST berdasarkan makna setiap node, tanpa terbebani detail tambahan dari parse tree.

```
struct AstNode {
    AstKind kind;
    std::string text;
    std::vector<AstNode> children;

    // Diisi pada semantic analysis/decorated AST.
    std::string inferredType;
    int symbolIndex;
    int lexicalLevel;
    ...
};
```

Snippet tersebut menunjukkan bahwa AstNode tidak hanya menyimpan struktur AST melalui kind, text, dan children, tetapi juga menyimpan field anotasi semantic. Field inferredType, symbolIndex, dan lexicalLevel digunakan setelah semantic analysis untuk membentuk Decorated AST. Struktur ini sesuai dengan implementasi pada ast.h.

Pada tahap konversi, detail sintaks yang tidak diperlukan untuk semantic analysis dapat dihilangkan, sedangkan elemen penting seperti deklarasi, statement, ekspresi, pemanggilan procedure/function, akses array, dan akses record tetap dipertahankan. Contohnya, assignment statement pada parse tree dikonversi menjadi node AST AssignStmt.

```
AstNode buildStatement(const ParseNode &node) {
    ...
    if (hasExactLabel(child, "<assignment-statement>")) {
        AstNode stmt(AstKind::AssignStmt);
        stmt.addChild(buildVariable(child.children.at(0)));
        stmt.addChild(buildExpression(child.children.at(2)));
        return stmt;
    }
    ...
}
```

Snippet tersebut menunjukkan salah satu bentuk semantic action pada AST builder. Node parse tree <assignment-statement> diubah menjadi node AST AssignStmt, lalu bagian variable dan expression disimpan sebagai child. Detail sintaks seperti token becomes tidak

dimasukkan ke AST karena tidak diperlukan lagi pada semantic analysis. Bagian ini diimplementasikan pada `ast_builder.cpp`.

Setelah AST terbentuk, semantic analyzer akan memberi anotasi pada node AST. AST yang sudah diberi anotasi ini disebut **Decorated AST**. Anotasi yang diberikan antara lain tipe data hasil node, indeks ke Symbol Table, dan lexical level. Dalam implementasi, informasi tersebut disimpan pada field `inferredType`, `symbolIndex`, dan `lexicalLevel` pada `AstNode`.

Decorated AST digunakan sebagai salah satu output utama Milestone 3 karena menunjukkan hasil semantic analysis yang lebih jelas. Melalui **Decorated AST**, pembaca tidak hanya dapat melihat susunan program, tetapi juga informasi semantik yang berhasil dikumpulkan oleh program, seperti tipe ekspresi dan hubungan node dengan Symbol Table.

G. Implementasi Symbol Table

Symbol Table digunakan untuk menyimpan informasi identifier yang ditemukan selama semantic analysis. Informasi ini diperlukan agar program dapat mengecek apakah suatu identifier sudah dideklarasikan, berada pada scope yang benar, dan memiliki tipe yang sesuai ketika digunakan.

Pada implementasi ini, **Symbol Table** dibuat dalam class `SymbolTable`. Class ini mengelola tiga tabel utama, yaitu `tab`, `btab`, dan `atab`.

- `tab` digunakan untuk menyimpan informasi identifier, seperti nama identifier, jenis objek, tipe data, lexical level, reference, address, dan link ke identifier lain dalam scope yang sama.
- `btab` digunakan untuk menyimpan informasi block atau scope, seperti program, procedure, function, record, dan compound block.
- `atab` digunakan untuk menyimpan informasi mengenai array, seperti tipe indeks array, tipe elemen array, batas bawah, batas atas, dan ukuran elemen.

```
class SymbolTable {
private:
    std::vector<TabEntry> tab_;
    std::vector<BTabEntry> btab_;
    std::vector<ATabEntry> atab_;
    std::vector<int> display_;
    int currentLevel_;
    ...
}
```

```
};
```

Snippet tersebut menunjukkan bahwa `SymbolTable` menyimpan tiga tabel utama, yaitu `tab_`, `btab_`, dan `atab_`. Selain itu, `display_` dan `currentLevel_` digunakan untuk membantu pengelolaan scope yang sedang aktif selama semantic analysis. Implementasi ini terdapat pada `symbol_table.h`.

Selain tiga tabel tersebut, `SymbolTable` juga menggunakan mekanisme scope stack/display untuk mencatat scope yang sedang aktif. Ketika semantic analyzer masuk ke block baru, seperti procedure, function, record, atau compound statement, program memanggil `pushScope`. Ketika keluar dari block tersebut, program memanggil `popScope`. Dengan cara ini, program dapat membedakan identifier global dan identifier lokal.

Proses pencarian identifier dilakukan dengan lookup. Pencarian dimulai dari scope terdalam, lalu bergerak ke scope luar. Pendekatan ini digunakan agar aturan scope dapat berjalan dengan benar. Jika terdapat identifier lokal dengan nama yang sama, identifier tersebut akan ditemukan lebih dulu dibandingkan identifier pada scope luar.

```
int SymbolTable::lookup(const std::string &name) const {
    const std::string normalized = normalizeIdentifier(name);
    for (int level = currentLevel_; level >= 0; --level) {
        int index = btab_[display_[level]].last;
        while (index > 0) {
            const TabEntry &entry = tab_[index];
            if (entry.identifier == normalized) {
                return index;
            }
            index = entry.link;
        }
    }
    return -1;
}
```

Snippet tersebut menunjukkan implementasi pencarian identifier pada `SymbolTable`. Pencarian dimulai dari `currentLevel_`, yaitu scope terdalam yang sedang aktif, lalu bergerak ke level luar. Pada setiap level, program mengambil identifier terakhir dari `btab_` melalui `display_`, kemudian menelusuri identifier dalam scope tersebut menggunakan field `link`. Jika identifier ditemukan, fungsi mengembalikan indeksinya pada `tab_`. Jika tidak ditemukan di semua scope, fungsi mengembalikan `-1`.

Proses penambahan identifier dilakukan dengan insert. Fungsi ini digunakan saat semantic analyzer menemukan deklarasi program, konstanta, variabel, tipe, procedure, function, parameter, atau field record. Untuk tipe array, informasi tambahan disimpan melalui `insertArray` ke dalam `atab`.

```
int SymbolTable::insertInternal(const std::string &name, ObjectKind obj,
                                TypeKind type, int ref, bool nrm, int adr,
                                bool initialized,
                                const std::string &literalValue,
                                bool checkRedeclaration) {
    ...
    const int blockIndex = currentBlock();
    const int previous = btab_[blockIndex].last;
    const int index = static_cast<int>(tab_.size());

    tab_.push_back(makeEntry(normalized, previous, obj, type, ref, nrm,
                              currentLevel_, adr, initialized, literalValue));
    btab_[blockIndex].last = index;
    ...
};
```

Snippet tersebut menunjukkan inti proses penambahan identifier pada `SymbolTable`. Program mengambil block yang sedang aktif melalui `currentBlock()`, lalu menyimpan indeks identifier terakhir pada scope tersebut ke variabel `previous`. Identifier baru kemudian dimasukkan ke `tab_`, dengan `previous` disimpan sebagai field `link`. Setelah itu, `btab_[blockIndex].last` diperbarui agar menunjuk ke identifier yang baru dimasukkan. Dengan cara ini, setiap scope memiliki daftar identifier sendiri yang dapat ditelusuri melalui link.

Perancangan dan implementasi **Symbol Table** seperti ini dipilih karena sesuai dengan kebutuhan Milestone 3 dan memisahkan informasi identifier, block, dan array ke tabel yang berbeda. Dengan pemisahan tersebut, semantic analyzer dapat melakukan pengecekan deklarasi, scope resolution, type checking, dan anotasi AST dengan lebih terarah.

H. Implementasi Semantic Analyzer

Semantic Analyzer dirancang untuk menelusuri AST dan memeriksa apakah program sudah benar secara semantik. Tahap ini tidak lagi memeriksa bentuk sintaks, karena hal tersebut sudah dilakukan oleh parser. Fokus **Semantic Analyzer** adalah memastikan penggunaan

identifier, tipe data, scope, assignment, expression, array, record, dan pemanggilan procedure/function sudah sesuai aturan.

Proses semantic analysis dijalankan melalui fungsi `analyzeSemantics`. Fungsi ini menerima AST, membuat objek `SemanticAnalyzer`, lalu menjalankan proses analisis untuk menghasilkan `SemanticResult`.

```
SemanticResult analyzeSemantics(AstNode ast) {
    SemanticAnalyzer analyzer(std::move(ast));
    return analyzer.analyze();
}

class SemanticAnalyzer {
public:
    SemanticResult analyze() {
        visitProgram(ast_);
        return {std::move(ast_), symbols_, diagnostics_};
    }
    ...
};
```

Snippet tersebut menunjukkan bahwa `analyzeSemantics` menjadi penghubung antara AST dan implementasi semantic analyzer. Fungsi ini menyembunyikan detail class `SemanticAnalyzer`, sehingga modul lain cukup memanggil satu fungsi untuk menjalankan semantic analysis. Implementasi ini terdapat pada `semantic_analyzer.cpp`.

Semantic Analyzer menggunakan pendekatan fungsi visit atau **Semantic Visitor**. Artinya, setiap jenis node AST diproses oleh fungsi yang sesuai dengan perannya. Misalnya, deklarasi diproses melalui fungsi `visit declaration`, statement diproses melalui fungsi `visit statement`, sedangkan expression diproses melalui fungsi `eval expression`. Pendekatan ini dipilih karena setiap jenis node memiliki aturan semantic yang berbeda.

```
bool visitStatement(AstNode &node) {
    node.lexicalLevel = symbols_.currentLevel();
    switch (node.kind) {
    case AstKind::CompoundStmt:
    case AstKind::StatementList: {
        bool assignedOnAllPaths = false;
        for (AstNode &child : node.children) {
            assignedOnAllPaths = visitStatement(child) ||
    }
```

```

assignedOnAllPaths;
    }
    annotate(node, TypeInfo{});
    return assignedOnAllPaths;
}
case AstKind::EmptyStmt:
    annotate(node, TypeInfo{});
    return false;
case AstKind::AssignStmt:
    return visitAssignment(node);
case AstKind::IfStmt:
    return visitConditional(node, "if");
case AstKind::WhileStmt:
    return visitLoop(node, "while");
case AstKind::RepeatStmt:
    return visitRepeat(node);
case AstKind::ForStmt:
    return visitFor(node);
case AstKind::CaseStmt:
    return visitCase(node);
case AstKind::Call:
    evalCall(node, true);
    return false;
case AstKind::Identifier:
    visitBareIdentifierStatement(node);
    return false;
default:
    evalExpression(node);
    return false;
}
}

```

Snippet tersebut menunjukkan pola utama **Semantic Visitor**. Fungsi `visitStatement` membaca jenis node AST, lalu mengarahkan proses ke fungsi yang sesuai.

Beberapa kelompok fungsi utama pada semantic analyzer adalah sebagai berikut.

1. **Visit untuk program dan deklarasi**

Fungsi seperti `visitProgram`, `visitDeclarations`, `visitConstDecl`,

`visitTypeDecl`, `visitVarDecl`, `visitProcedureDecl`, dan `visitFunctionDecl` digunakan untuk memproses bagian deklarasi. Pada tahap ini, identifier seperti konstanta, variabel, tipe, procedure, function, parameter, dan field record dimasukkan ke `Symbol Table`.

2. Visit untuk statement

Fungsi `visitStatement` mengarahkan proses pengecekan berdasarkan jenis statement, seperti assignment, if, while, repeat, for, case, dan pemanggilan procedure/function. Dari sini, `SemanticAnalyzer` dapat memeriksa aturan khusus untuk setiap statement.

3. Evaluasi expression dan variable

Fungsi seperti `evalExpression`, `evalIdentifier`, `evalVariable`, `evalUnary`, dan `evalBinary` digunakan untuk menentukan tipe hasil expression. Fungsi-fungsi ini juga mengecek apakah identifier sudah dideklarasikan, apakah identifier valid sebagai nilai, dan apakah operasi yang digunakan cocok dengan tipe operand.

```
ExprInfo evalExpression(AstNode &node) {
    switch (node.kind) {
        case AstKind::Literal:
            return evalLiteral(node);
        case AstKind::Identifier:
            return evalIdentifier(node, false);
        case AstKind::Variable:
            return evalVariable(node, false);
        case AstKind::UnaryExpr:
            return evalUnary(node);
        case AstKind::BinaryExpr:
            return evalBinary(node);
        case AstKind::Call:
            return evalCall(node, false);
        default:
            annotate(node, TypeInfo{});
            return {};
    }
}
```

Snippet tersebut menunjukkan bahwa `evalExpression` bertugas mengarahkan proses evaluasi expression berdasarkan jenis node AST. Literal, identifier, variable, unary expression, binary expression, dan call diproses oleh fungsi yang berbeda karena masing-masing

memiliki aturan semantic yang berbeda. Hasil evaluasi disimpan dalam `ExprInfo`, sehingga tipe expression dan informasi symbol table dapat digunakan oleh pengecekan semantic berikutnya.

4. Pengecekan call, array, dan record

Fungsi `evalCall` digunakan untuk mengecek pemanggilan procedure/function, termasuk jumlah dan tipe argumen. Fungsi `evalIndexAccess` digunakan untuk mengecek akses array, sedangkan `evalFieldAccess` digunakan untuk mengecek akses field pada record.

5. Pengecekan kompatibilitas tipe

Fungsi `typeCompatible` dan `assignmentCompatible` digunakan untuk memusatkan aturan kecocokan tipe. Dengan cara ini, pengecekan tipe untuk assignment, parameter function/procedure, operator, dan expression tidak perlu ditulis berulang di banyak tempat.

```
bool assignmentCompatible(const TypeInfo &target, const TypeInfo &value,
                          bool hasIntValue, int intValue) const {
    if (target.kind == TypeKind::None || value.kind == TypeKind::None) {
        return true;
    }
    if (target.kind == TypeKind::Real && underlying(value) ==
TypeKind::Integer) {
        return true;
    }
    if (target.kind == TypeKind::Subrange && hasIntValue &&
        (intValue < target.low || intValue > target.high)) {
        return false;
    }
    if (target.kind == TypeKind::Enum && target.hasRange && hasIntValue &&
        (intValue < target.low || intValue > target.high)) {
        return false;
    }
    if (typeCompatible(target, value) &&
        (isScalar(target.kind) || target.kind == TypeKind::String ||
         target.kind == TypeKind::Array || target.kind ==
TypeKind::Record)) {
        return true;
    }
}
```

```
    return false;
}
```

Snippet tersebut menunjukkan bahwa aturan assignment compatibility dipusatkan dalam satu fungsi. Contohnya, nilai integer dapat diberikan ke target real, sedangkan nilai literal untuk subrange harus berada dalam batas yang sesuai. Dengan pemisahan ini, aturan kompatibilitas tipe dapat dipakai kembali oleh assignment, parameter function/procedure, dan pengecekan semantic lainnya.

Selain aturan utama dari spesifikasi, `SemanticAnalyzer` juga menambahkan beberapa validasi tambahan untuk memperjelas hasil analisis. Contohnya, function dicek agar memberi nilai balik melalui assignment ke nama function, variabel yang mungkin digunakan sebelum inisialisasi diberi warning, dan literal index array dicek agar tidak keluar dari batas array. Validasi tambahan ini tidak mengubah grammar, tetapi membantu Semantic Diagnostics menjadi lebih informatif.

Setiap node yang selesai dianalisis akan diberi anotasi semantic, seperti tipe hasil, indeks Symbol Table, dan lexical level. Anotasi ini membuat AST berubah menjadi Decorated AST, sehingga hasil semantic analysis dapat dilihat langsung pada output program.

I. Implementasi Error Handling & Output

Error handling dirancang agar program tidak langsung crash ketika ketemu input atau proses yang bermasalah. Penanganan error dilakukan berdasarkan tahap pipeline, sehingga pesan error dapat menunjukkan bagian mana yang gagal, misalnya lexer, parser, AST builder, atau semantic analyzer.

Penanganan error pada program dibagi menjadi beberapa bagian berikut.

1. Error pembacaan file dan argumen untuk main

Program memeriksa jumlah argumen dan memastikan file input dapat dibuka. Jika argumen tidak sesuai atau file tidak dapat dibaca, program menampilkan pesan error dan proses dihentikan.

2. Error leksikal

Jika lexer menemukan token tidak valid, program menampilkan laporan error leksikal. Parsing kemudian dibatalkan karena token yang tidak valid dapat membuat hasil syntax analysis tidak dapat dipercaya.

3. Error parser dan AST builder

Jika terjadi syntax error atau kegagalan saat membentuk AST, program menangkap exception dan menampilkan pesan dengan penanda tahap, seperti [PARSER] atau

[AST]. Setelah itu, program dihentikan karena AST belum dapat digunakan untuk semantic analysis.

4. Semantic diagnostics

Pada tahap semantic analysis, error dan warning dikumpulkan dalam `SemanticDiagnostic`. Dengan cara ini, program dapat melaporkan lebih dari satu masalah semantik dalam satu kali proses analisis. Error digunakan untuk pelanggaran aturan semantik, sedangkan warning digunakan untuk kondisi yang mencurigakan tetapi tidak langsung menghentikan proses analisis.

```
struct SemanticDiagnostic {
    enum class Severity { Error, Warning };

    Severity severity;
    std::string message;
};

struct SemanticResult {
    AstNode ast;
    SymbolTable symbols;
    std::vector<SemanticDiagnostic> diagnostics;

    bool ok() const;
};
```

Snippet tersebut menunjukkan bahwa hasil semantic analysis tidak hanya berupa AST dan Symbol Table, tetapi juga daftar diagnostics. Setiap diagnostic memiliki tingkat severity dan pesan, sehingga semantic analyzer dapat membedakan error dan warning. Implementasi ini terdapat pada `semantic_analyzer.h`.

5. Status program

Hasil semantic analysis disimpan dalam `SemanticResult`. Jika terdapat diagnostic dengan severity tingkat error, fungsi `ok()` akan mengembalikan nilai false. Program tetap menampilkan hasil analisis, tetapi mengembalikan status gagal. Jika hanya terdapat warning atau tidak ada diagnostic, program dianggap berhasil.

```
bool SemanticResult::ok() const {
    for (const SemanticDiagnostic &diagnostic : diagnostics) {
        if (diagnostic.severity == SemanticDiagnostic::Severity::Error) {
```

```
        return false;
    }
}
return true;
}
```

Snippet tersebut menunjukkan bahwa status akhir semantic analysis ditentukan dari isi diagnostics. Jika terdapat error, hasil dianggap gagal secara semantik. Namun, output tetap dapat ditampilkan agar pengguna dapat melihat Decorated AST, Symbol Table, dan daftar error yang ditemukan. Implementasi ini terdapat pada `semantic_analyzer.cpp`.

Output program terdiri dari tiga bagian utama:

- **Decorated AST**, yaitu AST yang sudah diberi anotasi semantik.
- **Symbol Table**, yaitu hasil pencatatan identifier, scope, block, dan array.
- **Semantic Diagnostics**, yaitu daftar error atau warning yang ditemukan selama semantic analysis.

Pemisahan output menjadi tiga bagian dipilih agar hasil semantic analysis lebih mudah dibaca. **Decorated AST** menunjukkan hasil anotasi pada node, **Symbol Table** menunjukkan informasi identifier yang terkumpul, sedangkan **Semantic Diagnostics** membantu pengguna mengetahui masalah yang ditemukan oleh program.

Pengujian

Untuk memastikan bahwa semantic analyzer dapat mengenali seluruh komponen bahasa Arion dengan benar, dilakukan serangkaian pengujian menggunakan berbagai skenario input yang menghasilkan output.

Pengujian 1

Diberikan input sebagai berikut:

```
program ValidBasic;

const
  LIMIT == 10;
var
  x, y: integer;
  ok: boolean;

begin
  x := 3;
  y := x + LIMIT;
  ok := y > 5;
  if ok then
    writeln('valid')
  end.
end.
```

Dihasilkan output sebagai berikut:

```
----- Decorated AST -----

Program(ValidBasic) [type=none, sym=37, level=0]
|-- Declarations [type=none, level=0]
|   |-- ConstDecl(LIMIT) [type=integer, sym=38, level=0]
|   |   |-- Literal(10) [type=integer, level=0]
|   |-- VarDecl(x) [type=integer, sym=39, level=0]
|   |   |-- NamedType(integer) [type=integer, sym=22, level=0]
|   |-- VarDecl(y) [type=integer, sym=40, level=0]
|   |   |-- NamedType(integer) [type=integer, sym=22, level=0]
|   |-- VarDecl(ok) [type=boolean, sym=41, level=0]
|   |   |-- NamedType(boolean) [type=boolean, sym=24, level=0]
|-- CompoundStmt [type=none, level=1]
|   |-- AssignStmt [type=none, level=1]
|   |   |-- Variable(x) [type=integer, sym=39, level=1]
|   |   |-- Literal(3) [type=integer, level=1]
|   |-- AssignStmt [type=none, level=1]
|   |   |-- Variable(y) [type=integer, sym=40, level=1]
|   |   |-- BinaryExpr(plus) [type=integer, level=1]
|   |       |-- Identifier(x) [type=integer, sym=39, level=1]
|   |       |-- Identifier(LIMIT) [type=integer, sym=38, level=1]
```

```

|-- AssignStmt [type=none, level=1]
|   |-- Variable(ok) [type=boolean, sym=41, level=1]
|   |-- BinaryExpr(gtr) [type=boolean, level=1]
|       |-- Identifier(y) [type=integer, sym=40, level=1]
|       |-- Literal(5) [type=integer, level=1]
|-- IfStmt [type=none, level=1]
    |-- Identifier(ok) [type=boolean, sym=41, level=1]
    |-- Call(writeln) [type=none, sym=36, level=1]
        |-- Literal('valid') [type=string, level=1]

```

----- Symbol Table -----

tab

idx	identifier	obj	type	ref	nrm	lev	adr
link	init	value					
0	<dummy>	none	none	0	1	0	0
0	1						
1	and	reserved	none	0	1	0	0
0	1						
2	array	reserved	none	0	1	0	0
1	1						
3	begin	reserved	none	0	1	0	0
2	1						
4	case	reserved	none	0	1	0	0
3	1						
5	const	reserved	none	0	1	0	0
4	1						
6	div	reserved	none	0	1	0	0
5	1						
7	downto	reserved	none	0	1	0	0
6	1						
8	do	reserved	none	0	1	0	0
7	1						
9	else	reserved	none	0	1	0	0
8	1						
10	end	reserved	none	0	1	0	0
9	1						
11	for	reserved	none	0	1	0	0
10	1						
12	function	reserved	none	0	1	0	0
11	1						
13	if	reserved	none	0	1	0	0
12	1						
14	mod	reserved	none	0	1	0	0
13	1						

15	not	reserved	none	0	1	0	0
14	1						
16	of	reserved	none	0	1	0	0
15	1						
17	or	reserved	none	0	1	0	0
16	1						
18	procedure	reserved	none	0	1	0	0
17	1						
19	program	reserved	none	0	1	0	0
18	1						
20	record	reserved	none	0	1	0	0
19	1						
21	repeat	reserved	none	0	1	0	0
20	1						
22	integer	type	integer	0	1	0	0
21	1						
23	real	type	real	0	1	0	0
22	1						
24	boolean	type	boolean	0	1	0	0
23	1						
25	char	type	char	0	1	0	0
24	1						
26	string	type	string	0	1	0	0
25	1						
27	then	reserved	none	0	1	0	0
26	1						
28	to	reserved	none	0	1	0	0
27	1						
29	type	reserved	none	0	1	0	0
28	1						
30	until	reserved	none	0	1	0	0
29	1						
31	var	reserved	none	0	1	0	0
30	1						
32	while	reserved	none	0	1	0	0
31	1						
33	true	constant	boolean	0	1	0	1
32	1	true					
34	false	constant	boolean	0	1	0	0
33	1	false					
35	readln	procedure	none	0	1	0	0
34	1						
36	writeln	procedure	none	0	1	0	0
35	1						
37	validbasic	program	none	0	1	0	0
36	1						
38	limit	constant	integer	0	1	0	0
37	1	10					
39	x	variable	integer	0	1	0	0

```

38  1
40  y          variable  integer  0    1    0    0
39  1
41  ok          variable  boolean  0    1    0    0
40  1

btab
-----
idx  kind      last  lpar  psze  vsze
-----
0    program   41    0    0    3
1    compound  0     0    0    0

atab
-----
idx  xtyp      etyp      eref  low   high  elsz  size
-----

----- Semantic Diagnostics -----

No semantic diagnostics.

```

Pengujian 2

Diberikan input sebagai berikut:

```

program BadAssignment;

var
  x: integer;

begin
  x := 'hello'
end.

```

Dihasilkan output sebagai berikut:

```

----- Decorated AST -----

Program(BadAssignment) [type=none, sym=37, level=0]
|-- Declarations [type=none, level=0]
|  |-- VarDecl(x) [type=integer, sym=38, level=0]
|  |    |-- NamedType(integer) [type=integer, sym=22, level=0]
|-- CompoundStmt [type=none, level=1]
|  |-- AssignStmt [type=none, level=1]
|  |    |-- Variable(x) [type=integer, sym=38, level=1]
|  |    |-- Literal('hello') [type=string, level=1]

----- Symbol Table -----

```

tab

idx	identifier	obj	type	ref	nrm	lev	adr
link	init	value					
0	<dummy>	none	none	0	1	0	0
0	1						
1	and	reserved	none	0	1	0	0
0	1						
2	array	reserved	none	0	1	0	0
1	1						
3	begin	reserved	none	0	1	0	0
2	1						
4	case	reserved	none	0	1	0	0
3	1						
5	const	reserved	none	0	1	0	0
4	1						
6	div	reserved	none	0	1	0	0
5	1						
7	downto	reserved	none	0	1	0	0
6	1						
8	do	reserved	none	0	1	0	0
7	1						
9	else	reserved	none	0	1	0	0
8	1						
10	end	reserved	none	0	1	0	0
9	1						
11	for	reserved	none	0	1	0	0
10	1						
12	function	reserved	none	0	1	0	0
11	1						
13	if	reserved	none	0	1	0	0
12	1						
14	mod	reserved	none	0	1	0	0
13	1						
15	not	reserved	none	0	1	0	0
14	1						
16	of	reserved	none	0	1	0	0
15	1						
17	or	reserved	none	0	1	0	0
16	1						
18	procedure	reserved	none	0	1	0	0
17	1						
19	program	reserved	none	0	1	0	0
18	1						
20	record	reserved	none	0	1	0	0

19	1						
21	repeat	reserved	none	0	1	0	0
20	1						
22	integer	type	integer	0	1	0	0
21	1						
23	real	type	real	0	1	0	0
22	1						
24	boolean	type	boolean	0	1	0	0
23	1						
25	char	type	char	0	1	0	0
24	1						
26	string	type	string	0	1	0	0
25	1						
27	then	reserved	none	0	1	0	0
26	1						
28	to	reserved	none	0	1	0	0
27	1						
29	type	reserved	none	0	1	0	0
28	1						
30	until	reserved	none	0	1	0	0
29	1						
31	var	reserved	none	0	1	0	0
30	1						
32	while	reserved	none	0	1	0	0
31	1						
33	true	constant	boolean	0	1	0	1
32	1	true					
34	false	constant	boolean	0	1	0	0
33	1	false					
35	readln	procedure	none	0	1	0	0
34	1						
36	writeln	procedure	none	0	1	0	0
35	1						
37	badassignment	program	none	0	1	0	0
36	1						
38	x	variable	integer	0	1	0	0
37	1						

btab

idx	kind	last	lpar	psze	vsze
0	program	38	0	0	1
1	compound	0	0	0	0

atab

idx	xtyp	etyp	eref	low	high	elsz	size
-----	------	------	------	-----	------	------	------

```
----- Semantic Diagnostics -----
```

```
error: Incompatible assignment: cannot assign string to integer
```

Pengujian 3

Diberikan input sebagai berikut:

```
program UndeclaredUse;

var
  x: integer;

begin
  x := y + 1
end
```

Dihasilkan output sebagai berikut:

```
----- Decorated AST -----
```

```
Program(UndeclaredUse) [type=none, sym=37, level=0]
|-- Declarations [type=none, level=0]
|  \-- VarDecl(x) [type=integer, sym=38, level=0]
|    \-- NamedType(integer) [type=integer, sym=22, level=0]
|-- CompoundStmt [type=none, level=1]
|   \-- AssignStmt [type=none, level=1]
|       |-- Variable(x) [type=integer, sym=38, level=1]
|       |-- BinaryExpr(plus) [type=none, level=1]
|           |-- Identifier(y) [type=none, level=1]
|           |-- Literal(1) [type=integer, level=1]
```

```
----- Symbol Table -----
```

```
tab
```

idx	identifiser	obj	type	ref	nrm	lev	adr
link	init	value					
0	<dummy>	none	none	0	1	0	0
0	1						
1	and	reserved	none	0	1	0	0
0	1						
2	array	reserved	none	0	1	0	0
1	1						

3	begin	reserved	none	0	1	0	0
2	1						
4	case	reserved	none	0	1	0	0
3	1						
5	const	reserved	none	0	1	0	0
4	1						
6	div	reserved	none	0	1	0	0
5	1						
7	downto	reserved	none	0	1	0	0
6	1						
8	do	reserved	none	0	1	0	0
7	1						
9	else	reserved	none	0	1	0	0
8	1						
10	end	reserved	none	0	1	0	0
9	1						
11	for	reserved	none	0	1	0	0
10	1						
12	function	reserved	none	0	1	0	0
11	1						
13	if	reserved	none	0	1	0	0
12	1						
14	mod	reserved	none	0	1	0	0
13	1						
15	not	reserved	none	0	1	0	0
14	1						
16	of	reserved	none	0	1	0	0
15	1						
17	or	reserved	none	0	1	0	0
16	1						
18	procedure	reserved	none	0	1	0	0
17	1						
19	program	reserved	none	0	1	0	0
18	1						
20	record	reserved	none	0	1	0	0
19	1						
21	repeat	reserved	none	0	1	0	0
20	1						
22	integer	type	integer	0	1	0	0
21	1						
23	real	type	real	0	1	0	0
22	1						
24	boolean	type	boolean	0	1	0	0
23	1						
25	char	type	char	0	1	0	0
24	1						
26	string	type	string	0	1	0	0
25	1						
27	then	reserved	none	0	1	0	0


```

26  1
28  to          reserved  none    0      1      0      0
27  1
29  type        reserved  none    0      1      0      0
28  1
30  until       reserved  none    0      1      0      0
29  1
31  var         reserved  none    0      1      0      0
30  1
32  while       reserved  none    0      1      0      0
31  1
33  true        constant  boolean 0      1      0      1
32  1      true
34  false       constant  boolean 0      1      0      0
33  1      false
35  readln      procedure  none    0      1      0      0
34  1
36  writeln     procedure  none    0      1      0      0
35  1
37  undeclareduse program  none    0      1      0      0
36  1
38  x           variable  integer 0      1      0      0
37  1

```

btab

```

-----
idx  kind      last  lpar  psze  vsze
-----
0    program   38    0     0     1
1    compound   0     0     0     0

```

atab

```

-----
idx  xtyp      etyp      eref  low  high  elsz  size
-----

```

----- Semantic Diagnostics -----

```

error: Undeclared identifier: y
error: Operator plus requires numeric operands

```

Pengujian 4

Diberikan input sebagai berikut:

```

program BadArrayIndex;

var

```

```

    numbers: array[1..3] of integer;

begin
    numbers[4] := 99
end.

```

Dihasilkan output sebagai berikut:

```

----- Decorated AST -----

Program(BadArrayIndex) [type=none, sym=37, level=0]
|-- Declarations [type=none, level=0]
|   |-- VarDecl(numbers) [type=array, sym=38, level=0]
|   |   |-- ArrayType [type=array, level=0]
|   |   |   |-- RangeType [type=subrange, level=0]
|   |   |   |   |-- Literal(1) [type=integer, level=0]
|   |   |   |   |-- Literal(3) [type=integer, level=0]
|   |   |   |-- NamedType(integer) [type=integer, sym=22, level=0]
|   |-- CompoundStmt [type=none, level=1]
|   |   |-- AssignStmt [type=none, level=1]
|   |   |   |-- Variable(numbers) [type=integer, sym=38, level=1]
|   |   |   |   |-- IndexAccess [type=integer, level=1]
|   |   |   |   |   |-- Literal(4) [type=integer, level=1]
|   |   |   |   |-- Literal(99) [type=integer, level=1]

```

----- Symbol Table -----

tab

idx	identifier	obj	type	ref	nrm	lev	adr
link	init	value					
0	<dummy>	none	none	0	1	0	0
0	1						
1	and	reserved	none	0	1	0	0
0	1						
2	array	reserved	none	0	1	0	0
1	1						
3	begin	reserved	none	0	1	0	0
2	1						
4	case	reserved	none	0	1	0	0
3	1						
5	const	reserved	none	0	1	0	0
4	1						
6	div	reserved	none	0	1	0	0
5	1						
7	downto	reserved	none	0	1	0	0

6	1						
8	do	reserved	none	0	1	0	0
7	1						
9	else	reserved	none	0	1	0	0
8	1						
10	end	reserved	none	0	1	0	0
9	1						
11	for	reserved	none	0	1	0	0
10	1						
12	function	reserved	none	0	1	0	0
11	1						
13	if	reserved	none	0	1	0	0
12	1						
14	mod	reserved	none	0	1	0	0
13	1						
15	not	reserved	none	0	1	0	0
14	1						
16	of	reserved	none	0	1	0	0
15	1						
17	or	reserved	none	0	1	0	0
16	1						
18	procedure	reserved	none	0	1	0	0
17	1						
19	program	reserved	none	0	1	0	0
18	1						
20	record	reserved	none	0	1	0	0
19	1						
21	repeat	reserved	none	0	1	0	0
20	1						
22	integer	type	integer	0	1	0	0
21	1						
23	real	type	real	0	1	0	0
22	1						
24	boolean	type	boolean	0	1	0	0
23	1						
25	char	type	char	0	1	0	0
24	1						
26	string	type	string	0	1	0	0
25	1						
27	then	reserved	none	0	1	0	0
26	1						
28	to	reserved	none	0	1	0	0
27	1						
29	type	reserved	none	0	1	0	0
28	1						
30	until	reserved	none	0	1	0	0
29	1						
31	var	reserved	none	0	1	0	0
30	1						

```

32  while          reserved  none    0      1      0      0
31  1
33  true           constant  boolean 0      1      0      1
32  1      true
34  false          constant  boolean 0      1      0      0
33  1      false
35  readln         procedure none     0      1      0      0
34  1
36  writeln        procedure none     0      1      0      0
35  1
37  badarrayindex  program   none     0      1      0      0
36  1
38  numbers        variable  array   0      1      0      0
37  1

```

btab

```

-----
idx  kind      last  lpar  psze  vsze
-----
0    program   38    0     0     1
1    compound   0     0     0     0

```

atab

```

-----
idx  xtyp      etyp      eref  low  high  elsz  size
-----
0    integer   integer    0     1    3     1    3

```

----- Semantic Diagnostics -----

error: Array index literal is outside declared bounds

Pengujian 5

Diberikan input sebagai berikut:

```

program MissingReturn;

function choose(x: integer): integer;
begin
  if x > 0 then
    choose := 1
  end;

begin
  writeln(choose(5))
end.

```

Dihasilkan output sebagai berikut:

```
----- Decorated AST -----

Program(MissingReturn) [type=none, sym=37, level=0]
|-- Declarations [type=none, level=0]
|   |-- FunctionDecl(choose) [type=integer, sym=38, level=0]
|   |   |-- Parameters [type=none, level=1]
|   |   |   |-- ParameterDecl(x) [type=integer, sym=39, level=1]
|   |   |   |-- NamedType(integer) [type=integer, sym=22,
level=1]
|   |   |       |-- NamedType(integer) [type=integer, sym=22, level=0]
|   |   |       |-- Declarations [type=none, level=1]
|   |   |       |-- CompoundStmt [type=none, level=1]
|   |   |           |-- IfStmt [type=none, level=1]
|   |   |               |-- BinaryExpr(gtr) [type=boolean, level=1]
|   |   |                   |-- Identifier(x) [type=integer, sym=39, level=1]
|   |   |                   |-- Literal(0) [type=integer, level=1]
|   |   |               |-- AssignStmt [type=none, level=1]
|   |   |                   |-- Variable(choose) [type=integer, sym=38,
level=1]
|   |   |                   |-- Literal(1) [type=integer, level=1]
|   |   |-- CompoundStmt [type=none, level=1]
|   |       |-- Call(writeln) [type=none, sym=36, level=1]
|   |       |-- Call(choose) [type=integer, sym=38, level=1]
|   |       |-- Literal(5) [type=integer, level=1]
```

----- Symbol Table -----

tab

idx	identifier	obj	type	ref	nrm	lev	adr
link	init	value					
0	<dummy>	none	none	0	1	0	0
0	1						
1	and	reserved	none	0	1	0	0
0	1						
2	array	reserved	none	0	1	0	0
1	1						
3	begin	reserved	none	0	1	0	0
2	1						
4	case	reserved	none	0	1	0	0
3	1						
5	const	reserved	none	0	1	0	0
4	1						
6	div	reserved	none	0	1	0	0
5	1						

7	downto	reserved	none	0	1	0	0
6	1						
8	do	reserved	none	0	1	0	0
7	1						
9	else	reserved	none	0	1	0	0
8	1						
10	end	reserved	none	0	1	0	0
9	1						
11	for	reserved	none	0	1	0	0
10	1						
12	function	reserved	none	0	1	0	0
11	1						
13	if	reserved	none	0	1	0	0
12	1						
14	mod	reserved	none	0	1	0	0
13	1						
15	not	reserved	none	0	1	0	0
14	1						
16	of	reserved	none	0	1	0	0
15	1						
17	or	reserved	none	0	1	0	0
16	1						
18	procedure	reserved	none	0	1	0	0
17	1						
19	program	reserved	none	0	1	0	0
18	1						
20	record	reserved	none	0	1	0	0
19	1						
21	repeat	reserved	none	0	1	0	0
20	1						
22	integer	type	integer	0	1	0	0
21	1						
23	real	type	real	0	1	0	0
22	1						
24	boolean	type	boolean	0	1	0	0
23	1						
25	char	type	char	0	1	0	0
24	1						
26	string	type	string	0	1	0	0
25	1						
27	then	reserved	none	0	1	0	0
26	1						
28	to	reserved	none	0	1	0	0
27	1						
29	type	reserved	none	0	1	0	0
28	1						
30	until	reserved	none	0	1	0	0
29	1						
31	var	reserved	none	0	1	0	0

```

30  1
32  while          reserved  none    0    1    0    0
31  1
33  true          constant  boolean 0    1    0    1
32  1      true
34  false        constant  boolean 0    1    0    0
33  1      false
35  readln       procedure  none    0    1    0    0
34  1
36  writeln      procedure  none    0    1    0    0
35  1
37  missingreturn program   none    0    1    0    0
36  1
38  choose       function   integer 1    1    0    0
37  1
39  x            parameter  integer 0    1    1    0
0   1

```

btab

```

-----
idx  kind      last  lpar  psze  vsze
-----
0   program    38    0    0    0
1   function    39    39    1    0
2   compound    0     0    0    0

```

atab

```

-----
idx  xtyp      etyp      eref  low  high  elsz  size
-----

```

----- Semantic Diagnostics -----

error: Function choose may not assign a return value on all control-flow paths

Kesimpulan dan Saran

Berdasarkan hasil pengembangan dan pengujian yang telah dilakukan pada milestone ini, bagian ini merangkum pencapaian fungsionalitas program serta memberikan rekomendasi untuk optimasi pada milestone pengembangan selanjutnya.

Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, program pada Milestone 3 telah berhasil mengimplementasikan tahap **semantic analysis** pada Arion Compiler. Tahap ini melanjutkan hasil dari lexer dan parser dengan membentuk **Abstract Syntax Tree (AST)**, lalu melakukan pengecekan semantik untuk memastikan bahwa program tidak hanya benar secara sintaksis, tetapi juga valid secara makna.

Secara umum, hasil yang diperoleh dari Milestone 3 adalah sebagai berikut:

1. Program mampu mengubah parse tree menjadi **AST** yang lebih ringkas dan lebih sesuai untuk proses semantic analysis.
2. Program mampu menghasilkan beberapa keluaran utama, yaitu:
 - Decorated AST, yang berisi AST dengan anotasi semantik;
 - Symbol Table, yang menyimpan informasi identifier, scope, block, dan array;
 - Semantic Diagnostics, yang menampilkan error atau warning semantik.
3. Semantic analyzer telah mampu melakukan beberapa pengecekan penting, seperti:
 - penggunaan identifier yang belum dideklarasikan,
 - deklarasi ulang pada scope yang sama,
 - kompatibilitas tipe data,
 - validasi assignment,
 - akses array,
 - pemanggilan procedure/function,
 - serta pengecekan ekspresi pada struktur kontrol.

Dari hasil pengujian, program dapat membedakan input yang valid dan input yang memiliki kesalahan semantik. Beberapa kesalahan seperti assignment dengan tipe yang tidak sesuai, penggunaan identifier yang belum dideklarasikan, indeks array di luar batas, dan function yang tidak selalu mengembalikan nilai berhasil dideteksi oleh program. Dengan demikian, implementasi pada Milestone 3 telah memenuhi tujuan utama semantic analysis dalam proses kompilasi.

Saran

Meskipun program sudah mampu melakukan semantic analysis dasar dengan baik, masih terdapat beberapa bagian yang dapat dikembangkan lebih lanjut agar compiler menjadi lebih lengkap dan mudah digunakan.

Beberapa saran pengembangan yang dapat dilakukan adalah sebagai berikut:

1. **Pengecekan semantik dapat diperluas**, terutama untuk kasus yang lebih kompleks seperti record bersarang, array multidimensi, nested scope, serta kombinasi procedure dan function yang lebih beragam.
2. **Pesan error dan warning dapat dibuat lebih informatif** dengan menambahkan informasi posisi kesalahan, seperti nomor baris dan kolom pada source code. Hal ini akan membantu pengguna menemukan sumber kesalahan dengan lebih cepat.
3. **Pengujian dapat diperbanyak** dengan skenario yang lebih bervariasi, misalnya:
 - deklarasi ulang identifier,
 - pemanggilan function dengan argumen yang salah,
 - penggunaan variabel dari scope luar,
 - akses field record yang tidak valid,
 - serta beberapa error dalam satu program.
4. **Tampilan Symbol Table dapat dirapikan kembali** agar informasi pada `tab`, `btab`, dan `atab` lebih mudah dibaca, terutama ketika program yang dianalisis memiliki banyak identifier dan scope.

Dengan pengembangan tersebut, semantic analyzer dapat menjadi lebih kuat dan siap digunakan sebagai dasar untuk tahap compiler berikutnya, seperti intermediate code generation atau proses translasi lanjutan.

Lampiran

Link Github: <https://github.com/raulinn/ISL-Tubes-IF2224-2026>

Pembagian Tugas:

NIM	Nama	Bagian	Persentase Keseluruhan
13524044	Narendra Dharma Wistara M	Symbol Table (<code>tab</code> , <code>bt</code> , <code>atab</code>), scope stack, predefined identifier, serta fungsi <code>lookup</code> , <code>insert</code> , <code>push</code> , dan <code>pop</code> .	25%
13524060	Steven Tan	Semantic visitor, integrasi ke <code>main</code> , output Decorated AST dan Symbol Table, pengujian end-to-end, serta error reporting.	25%
13524090	Nashiruddin Akram	Abstract Syntax Tree, definisi node AST, serta konversi parse tree menjadi AST.	25%
13524100	Arghawisesa Dwinanda Arham	Aturan semantik dan type checking, meliputi ekspresi, assignment, kondisi, array index, dan kompatibilitas tipe.	25%

Referensi

- [1] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to automata theory, languages, and computation*, 3. ed. Boston San Francisco Munich: Pearson, 2007.
- [2] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, & Tools*, 2nd ed. Boston, MA, USA: Pearson/Addison-Wesley, 2007.
- [3] K. D. Cooper and L. Torczon, *Engineering a Compiler*, 3rd ed. Cambridge, MA, USA: Morgan Kaufmann, 2022.
- [4] Laboratorium Ilmu Rekayasa dan Komputasi, "Spesifikasi Milestone 3 - Tubes IF2224 TBFO", Institut Teknologi Bandung, Bandung, 2026. [Online]. Available: [Spesifikasi Milestone 3 - Tubes IF2224 TBFO](#)